



LABORATORIO 2

MÉTODOS DE ORDENAMIENTO Y BÚSQUEDA

Documento 3 · Interfaz web: HTML, CSS y JavaScript

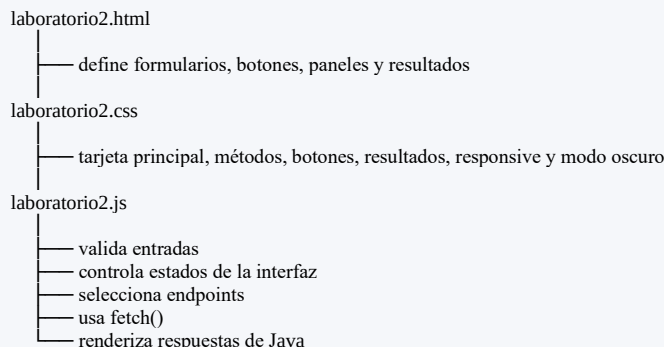
Contenido

- 1 Introducción
- 2 Arquitectura del frontend
- 3 HTML
- 4 CSS
- 5 JavaScript
- 6 Selección de métodos
- 7 Flujo Tamaño → Aceptar
- 8 Ordenar
- 9 Comparación de arreglos
- 10 Búsqueda binaria
- 11 Validaciones
- 12 Comunicación con Java
- 13 Modo oscuro y responsive
- 14 Flujo de usuario
- 15 Conclusiones

1. Introducción

La interfaz web del Laboratorio 2 permite ingresar un arreglo, seleccionar un algoritmo, ordenar y realizar búsqueda binaria sin trasladar la lógica de los algoritmos al navegador. HTML define la estructura, CSS presenta la interfaz y JavaScript coordina eventos, validaciones y solicitudes HTTP hacia Java.

2. Arquitectura del frontend



3. HTML

El HTML contiene la cabecera del laboratorio, botón Salir, formulario de entrada, menú de métodos, panel de comparación y panel de búsqueda. Los IDs son importantes porque JavaScript obtiene los elementos con getElementById.

Elementos principales

Elemento	ID / clase	Función
Tamaño	lab2-array-size	Cantidad de elementos
Números enteros	lab2-array-values	Contenido del arreglo
Aceptar / Modificar	lab2-size-toggle-button	Confirma o reinicia el tamaño
Ordenar	lab2-sort-button	Ejecuta executeSort
Buscar	lab2-search-button	Abre/cierra búsqueda
Resultados	lab2-results-panel	Comparación original/ordenado
Búsqueda	lab2-search-panel	Formulario de búsqueda binaria

Fragmento representativo

```
<label for="lab2-array-size">Tamaño del arreglo</label>
<input id="lab2-array-size" name="arraySize" type="number" min="1" step="1" inputmode="numeric" placeholder="Ejemplo: 8">
<button id="lab2-size-toggle-button" class="lab2-button lab2-button--primary" type="button">
<label for="lab2-array-values">Números enteros</label>
<textarea id="lab2-array-values" name="arrayValues" rows="5" spellcheck="false" aria-describedby="lab2-array-help"
placeholder="Ejemplo: 14, 3, 27, 8, 1" disabled></textarea>
<button id="lab2-sort-button" class="lab2-button lab2-button--primary" type="button">
```

```
<button id="lab2-search-button" class="lab2-button lab2-button--primary" type="button" aria-controls="lab2-search-panel" aria-expanded="false">
  <section id="lab2-results-panel" class="lab2-results" aria-labelledby="lab2-results-title" hidden>
  <section id="lab2-search-panel" class="lab2-search-panel" aria-labelledby="lab2-search-title" hidden>
```

4. CSS

CSS mantiene la identidad visual azul de la UNI y organiza la interfaz en tarjetas. Los resultados utilizan casillas individuales para cada número, colores suaves para diferenciar original y ordenado, y flex-wrap para evitar desbordamientos.

- Azul como color principal para títulos, líneas y botones.
- Tarjetas blancas con bordes redondeados y sombra suave.
- Arreglo desordenado: acentos rojos/rosados.
- Arreglo ordenado: acentos verdes.
- Botones grandes y estados disabled visibles.
- Media queries para adaptar columnas y controles en pantallas pequeñas.
- Reglas de modo oscuro para mantener contraste.

```
.lab2-results,
.lab2-search-panel {
  .lab2-results[hidden],
  .lab2-search-panel[hidden] {
    .lab2-results-heading {
      .lab2-results-heading {
        .lab2-results-heading h2 {
          .lab2-search-close {
            .lab2-search-close:hover {
              .lab2-search-close:active {
                .lab2-search-close:focus-visible,
                .lab2-results,
                .lab2-search-panel {
                  .lab2-results {
                    .lab2-results-heading {
                      .lab2-results-heading h2 {
                        .lab2-array-boxes {
                          .lab2-array-box {
                            .lab2-array-box:hover {
                              .lab2-array-box--unsorted {
                                .lab2-array-box--ordered {
                                  .lab2-search-panel {
                                    .lab2-search-mode {
                                      .lab2-search-mode legend {
                                        .lab2-search-options {
                                          .lab2-search-options label {
                                            .lab2-search-options input {
                                              .lab2-search-options span {
                                                .lab2-search-options input:checked + span {
                                                  .lab2-search-options input:focus-visible + span {
                                                    .lab2-search-actions {
                                                      .lab2-search-result {
                                                        .lab2-search-row {
                                                          .lab2-search-row span {
                                                            .lab2-search-row strong {
                                                              .lab2-search-message {
                                                                .lab2-search-message--success {
                                                                  .lab2-search-message--error {
                                                                    @media (max-width: 840px) {
                                                                      @media (max-width: 600px) {
                                                                        .lab2-results,
                                                                        .lab2-search-panel {
                                                                          .lab2-search-row {
                                                                            .lab2-search-actions .lab2-button {
                                                                              @media (prefers-reduced-motion: reduce) {
                                                                                .lab2-search-close,
                                                                                .lab2-search-panel {
                                                                                  html.dark-mode .lab2-page,
```

```
body.dark-mode.lab2-page {
html.dark-mode .lab2-input-panel,
html.dark-mode .lab2-results,
html.dark-mode .lab2-search-panel,
html.dark-mode .lab2-methods-panel {
html.dark-mode .lab2-methods-panel::before {
html.dark-mode .lab2-panel-heading,
html.dark-mode .lab2-section-heading,
html.dark-mode .lab2-results-heading {
html.dark-mode .lab2-panel-heading h2,
html.dark-mode .lab2-section-heading h2,
html.dark-mode .lab2-results-heading h2 {
html.dark-mode .lab2-eyebrow {
html.dark-mode .lab2-field label {
html.dark-mode .lab2-field input,
html.dark-mode .lab2-field textarea {
html.dark-mode .lab2-field input:focus,
html.dark-mode .lab2-field textarea:focus {
html.dark-mode .lab2-field input:disabled,
html.dark-mode .lab2-field textarea:disabled {
html.dark-mode .lab2-field input::placeholder,
html.dark-mode .lab2-field textarea::placeholder {
html.dark-mode .lab2-help {
html.dark-mode .lab2-menu-toggle {
html.dark-mode .lab2-menu-toggle:hover,
html.dark-mode .lab2-menu-toggle[aria-expanded="true"] {
html.dark-mode .lab2-search-close {
html.dark-mode .lab2-search-close:hover {
html.dark-mode .lab2-method-button {
html.dark-mode .lab2-method-button:hover {
html.dark-mode .lab2-method-button--selected {
html.dark-mode .lab2-selected-method {
```

5. JavaScript

El archivo laboratorio2.js está envuelto en una función autoejecutable y utiliza modo estricto. Define la URL base del servidor, las rutas de cada método, referencias al DOM y funciones para validar, enviar y renderizar datos.

```
const SERVER_BASE_URL = "http://localhost:8080";

const METHOD_CONFIG = Object.freeze({
  Burbuja: { backendValue: "burbuja", route: "/lab2/burbuja" },
  "Burbuja con señal": { backendValue: "burbuja-senal", route: "/lab2/burbuja-senal" },
  Sacudida: { backendValue: "sacudida", route: "/lab2/sacudida" },
  Baraja: { backendValue: "baraja", route: "/lab2/baraja" },
  Selección: { backendValue: "seleccion", route: "/lab2/seleccion" },
  Shell: { backendValue: "shell", route: "/lab2/shell" }
});
```

6. Selección de métodos

El menú permite escoger un método. selectMethod(...) actualiza el nombre seleccionado, marca visualmente el botón y cierra el panel. getSelectedMethodConfig() traduce el nombre visible a la ruta y al valor que entiende el backend.

7. Flujo Tamaño → Aceptar

1. Al iniciar, el campo Tamaño está disponible.
2. El área Números enteros permanece deshabilitada.
3. El usuario escribe un tamaño mayor que cero.
4. Aceptar o la tecla Enter ejecutan confirmArraySize().
5. arraySizeConfirmed pasa a true.

6. El tamaño queda bloqueado y se habilita Números enteros.
7. El botón cambia de Aceptar a Modificar arreglo.
8. Modificar arreglo cancela solicitudes, limpia resultados y regresa al estado inicial.

```
function updateArrayControls() {  
  arraySize.disabled = requestBusy || arraySizeConfirmed;  
  arrayValues.disabled = requestBusy || !arraySizeConfirmed;  
  // cambia Aceptar ↔ Modificar arreglo  
}
```

8. Ordenar

executeSort() valida método y arreglo, inicia una petición, llama al endpoint correspondiente y, si la respuesta contiene los arreglos esperados, ejecuta showArrays(data.original, data.ordenado).

```
async function executeSort() {  
  const methodConfig = requireSelectedMethod();  
  if (!methodConfig) return;  
  
  const arrayInput = readArrayInput(true);  
  if (arrayInput.message) return;  
  
  const controller = beginRequest("sort");  
  if (!controller) return;  
  
  try {  
    const data = await postForm(  
      methodConfig.route,  
      { tam: arrayInput.tam, elementos: arrayInput.elementos },  
      controller.signal  
    );  
  
    validateServerArrays(data);  
    showArrays(data.original, data.ordenado);  
    resultsPanel.hidden = false;  
  } finally {  
    finishRequest(controller);  
  }  
}
```

Corrección importante: El diseño nuevo usa unsortedBoxes, sortedBoxes y showArrays(...). No debe depender de variables antiguas como unsortedResult o sortedResult.

9. Comparación de arreglos

renderArrayBoxes(...) crea una casilla div por cada número. showArrays(...) llama a esa función para original y ordenado y actualiza Tamaño e Índices.

```
function renderArrayBoxes(container, values, type) {  
  container.innerHTML = "";  
  
  values.forEach((val) => {  
    const box = document.createElement("div");  
    box.className = `lab2-array-box lab2-array-box--${type}`;  
    box.textContent = String(val);  
    container.appendChild(box);  
  });  
}  
  
function showArrays(unsorted, sorted) {  
  renderArrayBoxes(unsortedBoxes, unsorted, "unsorted");  
  renderArrayBoxes(sortedBoxes, sorted, "ordered");  
}
```

```
}
```

10. Búsqueda binaria

El panel de búsqueda permite dos modos: por número y por índice. `updateSearchMode()` cambia etiquetas, placeholder y textos de ayuda. `executeSearch()` envía el arreglo, método seleccionado, tipo y valor a `/lab2/binaria`.

Buscar por número

El usuario introduce un número. La respuesta puede mostrar la posición en el arreglo desordenado y la posición en el ordenado. Si no existe, se ocultan los valores y se presenta el mensaje de no encontrado.

Buscar por índice

El usuario introduce una posición válida desde 0 hasta tamaño-1. La respuesta muestra el valor que ocupa esa posición en el arreglo original y en el arreglo ordenado.

```
const data = await postForm(
  "/lab2/binaria",
  {
    tam: arrayInput.tam,
    elementos: arrayInput.elementos,
    metodo: methodConfig.backendValue,
    tipo: SEARCH_TYPE_CONFIG[searchMode],
    valor: searchInput.value
  },
  controller.signal
);
```

11. Validaciones

- El tamaño debe ser un entero mayor que cero.
- El tamaño debe confirmarse antes de ingresar el arreglo.
- El arreglo no puede estar vacío.
- Los elementos deben ser enteros separados por comas.
- La cantidad de elementos debe coincidir exactamente con el tamaño.
- No se permiten valores repetidos.
- En búsqueda por índice, el valor debe estar entre 0 y tamaño - 1.

12. Comunicación con Java

`postForm(...)` crea `URLSearchParams`, envía POST con Content-Type `application/x-www-form-urlencoded`, espera texto, lo convierte a JSON y comprueba `response.ok` y `data.ok`. Si fetch falla por conexión se genera un mensaje específico para indicar que `Servidor.java` debe estar ejecutándose.

```
response = await fetch(`${SERVER_BASE_URL}${route}`, {
  method: "POST",
  headers: {
    "Content-Type": "application/x-www-form-urlencoded"
  },
  body,
  signal
});

const responseText = await response.text();
const data = JSON.parse(responseText);
```

13. Modo oscuro y responsive

El diseño debe mantener legibilidad tanto en modo claro como oscuro. En pantallas pequeñas las tarjetas de comparación pueden pasar de dos columnas a una; las casillas de los arreglos usan flex-wrap; y los botones pueden reorganizarse sin perder funcionalidad.

14. Flujo de usuario

9. Abrir Laboratorio 2.
10. Seleccionar un método de ordenamiento.
11. Escribir y confirmar el tamaño.
12. Ingresar números enteros separados por comas.
13. Pulsar Ordenar.
14. Revisar arreglo desordenado y ordenado en casillas.
15. Pulsar Buscar.
16. Elegir Por número o Por índice.
17. Ingresar el valor y ejecutar la búsqueda.
18. Revisar el resultado devuelto por Java.
19. Usar Modificar arreglo para iniciar un nuevo caso.

15. Conclusiones

La interfaz mantiene una separación clara entre presentación y procesamiento. HTML y CSS construyen la experiencia visual; JavaScript administra estados, validaciones y comunicación; Java ejecuta los algoritmos. Esta separación permite mejorar el diseño sin reescribir los métodos de ordenamiento y facilita depurar errores al identificar si pertenecen al DOM, a fetch o al backend.